COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/16/2026	**DATE SUBMITTED:**	
NAME:	Jarel Dave R. Egar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

In this activity, we will explore static and dynamic memory allocation through by utilizing arrays and other components mentioned beforehand, this will be for the implementation of a student list with the students' names and age. To begin, we will include pertinent libraries for input/output stream and strings (which we will be using in this activity).

```
#include <iostream>
#include <string.h>
```

Now we will create the student class with private attributes studentName and studentAge.

```
class Student{
private:
std::string studentName;
int studentAge;
```

14

Then, as discussed beforehand, we have to define the constructor and the big three: the destructor, copy constructor and copy assignment operator. We will assign default values for the parameters of our constructor in the event that it is uninitialized.

```
public:
//constructor
Student(std::string newName ="John Doe", int newAge=18){
studentName = std::move(newName);
studentAge = newAge;
std::cout << "Constructor Called." << std::endl;
};
```

The big three is then defined. We will add an output stream for each to observe when each of the following functions are implicitly or explicitly called.

```
//deconstructor
~Student(){
std::cout << "Destructor Called." << std::endl;
}
//Copy Constructor
Student(const Student &copyStudent){
std::cout << "Copy Constructor Called" << std::endl;
studentName = copyStudent.studentName;
studentAge = copyStudent.studentAge;
}
//Display Attributes
```

```
void printDetails(){
std::cout << this->studentName << " " << this->studentAge << std::endl;
}
};
```

The driver program is now to be defined. We want to utilize the main function to simply show the static and dynamic allocation. However, we will explore the initial goal of creating instances of our Student class.

```
int main() {
Student student1("Roman", 28);
Student student2(student1);
Student student3;
student3 = student2;
return 0;
}
```

Run the code and show the output. Include the screenshot in table 2-1 followed by your observation.

Now, modify the driver program so that we have the array size of 5, an array of Student objects, the list of students' names and their age.

```
int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};
15
return 0;
}
```

Run the code with the modified driver function and show the output. Include the screenshot in table 2-2 followed by your observation.

We have so far created static memory allocation through the use of the arrays. We will now dynamically allocate instances of the student class and store the newly created objects in the locations pointed to by our array.

```
int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};
for(int i = 0; i < j; i++){ //loop A
Student *ptr = new Student(namesList[i], ageList[i]);
studentList[i] = *ptr;
}
for(int i = 0; i < j; i++){ //loop B
studentList[i].printDetails();
}
return 0;
}
```

Discuss what is done by loop A and loop B in table 2-3. Additionally, discuss the output and whether the functions are working as intended. If any corrections were made, further provide your modification and analysis in table 2-4.

screenshot	
observation	When the program runs, it initializes an array of five Student objects, executing the default constructor five times to populate them with default values. Although custom data arrays and a copy constructor are defined, they remain completely idle since no custom information is ever assigned or cloned. Finally, upon exiting, the program automatically cleans up all five profiles from memory, running the destructor five times.

```
R_HOA2.1 > Egar_HOA2.1.cpp > Student
#include <iostream>
#include <string.h>


class Student{
private:
    std::string studentName;
    int studentAge;

public:
//Constructor
Student(std::string newName = "John Doe", int newAge = 18) {
    studentName = newName;
    studentAge = newAge;
    std::cout << "Constructor was called" << std::endl;
}
//Destructor
~Student(){
    std::cout <<"Destructor was called" <<std::endl;
}


//Copy Constructor
Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
```

```
//Display attributes
void printDetails(){

    std::cout << this->studentName << " " << this->studentAge << std::endl;

}

};


int main() {
    Student student1("Ja",19);
    Student student2(student1);
    Student student3;

return 0;
}
```

Screenshot



```
#include <iostream>
#include <string.h>


class Student{
private:
    std::string studentName;
    int studentAge;

public:
//Constructor
Student(std::string newName = "John Doe", int newAge = 18) {
    studentName = newName;
    studentAge = newAge;
    std::cout << "Constructor was called" << std::endl;
}
//Destructor
~Student(){
    std::cout <<"Destructor was called" <<std::endl;
}


//Copy Constructor
Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
```



```
//Display attributes
void printDetails(){

    std::cout << this->studentName << " " << this->studentAge << std::endl;

}

};


int main() {
   const size_t j= 5;

       Student studentList[j] = { };
       std::string nameList[j] = {"Ja", "Ralph", "Drew", "Gela", "Dane"};
       int ageList[j] = {19, 18, 20, 18, 19};

   return 0;
   }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\TIPQC\Documents\Egar_CPE21S1> & 'c:\Users\TIPQC\.vscode\extensions\ms-vscode.cpptools-1.22.2-win32-x64\debu
rosoft-MIEngine-Out-flohnbm2.ljh' '--stderr=Microsoft-MIEngine-Error-ms52b2jm.vlz' '--pid=Microsoft-MIEngine-Pid-iibzdi
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
PS C:\Users\TIPQC\Documents\Egar_CPE21S1>
```

Observation

The program instantly initializes five default students named "John Doe" aged 18, which triggers the constructor five times. However, because your custom lists of names and ages are never actually linked to the students, the array remains stuck with this unedited default data. Finally, since no students are duplicated, the copy constructor is completely ignored before the program exits and runs the destructor five times to clean up memory.

Screenshot	



```cpp
#include <iostream>
#include <string.h>


class Student{
private:
    std::string studentName;
    int studentAge;

public:
//Constructor
Student(std::string newName = "John Doe", int newAge = 18) {
    studentName = newName;
    studentAge = newAge;
    std::cout << "Constructor was called" << std::endl;
}
//Destructor
~Student(){
    std::cout <<"Destructor was called" <<std::endl;
}


//Copy Constructor
Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
```



```cpp
//Display attributes
void printDetails(){

    std::cout << this->studentName << " " << this->studentAge << std::endl;

}

};


int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};


    for(int i = 0; i < j; i++){ //Loop A
    Student *ptr = new Student(namesList[i], ageList[i]);
    studentList[i] = *ptr;

    }

    for(int i = 0; i < j; i++){ //Loop B
    studentList[i].printDetails();
    }

return 0;
}
```

Observation	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List
Apple PHP 10 x7
Banana PHP 10 x8
Broccoli PHP 60 x12
Lettuce PHP 50 x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.
Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.
Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.
Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.
Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

CODE

Egar_HOA2-1Supple.cpp | HOA2.1 2-2.cpp

EGAR_HOA2.1 > Egar_HOA2-1Supple.cpp > ...

```cpp
#include <iostream>
#include <string>

// class for fruits and vegies
// problem 1
class fruits {
private:
    std::string fruitname; // declare string fruitname
    double fruitprice;     // declare fruit price
    int fruitquantity;     // declare fruit quantity

public: // blueprint
    // constructor
    fruits(std::string newFruitName = "unknown", double newFruitPrice = 0.0, int newFruitQuantity = 0) {
        // methods for getting the fruit infos
        fruitname = std::move(newFruitName);
        fruitprice = std::move(newFruitPrice);
        fruitquantity = std::move(newFruitQuantity);

        std::cout << "The constructor is called." << std::endl;
    }

    // method to calculate total fruit price
    double CalcTotalfruitPrice() const { return fruitprice * fruitquantity; }

    // deconstructor
    ~fruits() {
        std::cout << "The Deconstuctor is called." << std::endl;
    }

    // copy constructor
    fruits(const fruits &copyfruit) {
        std::cout << "Copy constructor called" << std::endl;
        fruitname = copyfruit.fruitname;
        fruitprice = copyfruit.fruitprice;
        fruitquantity = copyfruit.fruitquantity;
    }
```

```
    // copy assignment operator
    fruits& operator=(const fruits &copyfruit) {
        std::cout << "Copy assignment operator called" << std::endl;
        if (this != &copyfruit) {
            fruitname = copyfruit.fruitname;
            fruitprice = copyfruit.fruitprice;
            fruitquantity = copyfruit.fruitquantity;
        }
        return *this;
    }

    // This getters are used to access the private members of the class
    std::string getName() const { return fruitname; }
    double getPrice() const { return fruitprice; }
    int getQuantity() const { return fruitquantity; }
};

class vegies {
private:
    std::string vegiename; // declare string vegiename
    double vegieprice;     // declare vegie price
    int vegiequantity;     // declare vegie quantity

public: // blueprint for vegies
    // constructor
    vegies(std::string newVegieName = "unknown", double newVegiePrice = 0.0, int newVegieQuantity = 0) {
        // methods for getting the vegie infos
        vegiename = std::move(newVegieName);
        vegieprice = std::move(newVegiePrice);
        vegiequantity = std::move(newVegieQuantity);

        std::cout << "The constructor is called." << std::endl;
    }

    // method to calculate total vegie price
    double CalcTotalvegiePrice() const { return vegieprice * vegiequantity; }

```

```
76      // deconstructor
77      ~vegies() {
78          std::cout << "The Deconstuctor is called." << std::endl;
79      }
80
81      // copy constructor
82      vegies(const vegies &copyvegie) {
83          std::cout << "Copy constructor called" << std::endl;
84          vegiename = copyvegie.vegiename;
85          vegieprice = copyvegie.vegieprice;
86          vegiequantity = copyvegie.vegiequantity;
87      }
88
89      // copy assignment operator
90      vegies& operator=(const vegies &copyvegie) {
91          std::cout << "Copy assignment operator called" << std::endl;
92          if (this != &copyvegie) {
93              vegiename = copyvegie.vegiename;
94              vegieprice = copyvegie.vegieprice;
95              vegiequantity = copyvegie.vegiequantity;
96          }
97          return *this;
98      }
99
100     // This getters are used to access the private members of the class
101     std::string getName() const { return vegiename; }
102     double getPrice() const { return vegieprice; }
103     int getQuantity() const { return vegiequantity; }
104 };
```

```cpp
// identify whether the item is a fruit or a vegie
enum itemtype { is_fruit, is_vegie };

struct groceryitem {
    itemtype type;
    fruits* fruitptr = nullptr; //pointer to fruits object
    vegies* vegieptr = nullptr; //pointer to vegies object
};
// problem 3 calculate the total sum of all items in the grocery list
double TotalSum(groceryitem list[], int listsize) {
    double totalsum = 0.0;
    for (int i = 0; i < listsize; ++i) {
        if (list[i].type == is_fruit && list[i].fruitptr != nullptr) {
            totalsum += list[i].fruitptr->CalcTotalfruitPrice();
        }
        else if (list[i].type == is_vegie && list[i].vegieptr != nullptr) {
            totalsum += list[i].vegieptr->CalcTotalvegiePrice();
        }
    }
    return totalsum;
}

// Function to print details of all items
void DisplayGroceryList(groceryitem list[], int listsize) {
    std::cout << "\n GROCERY LIST \n";
    for (int i = 0; i < listsize; ++i) {
        if (list[i].type == is_fruit && list[i].fruitptr != nullptr) {
            std::cout << "[Fruit] " << list[i].fruitptr->getName()
                      << " | PHP " << list[i].fruitptr->getPrice()
                      << " | Quantity: x" << list[i].fruitptr->getQuantity()
                      << " | Subtotal: PHP " << list[i].fruitptr->CalcTotalfruitPrice() << std::endl;
        }
        else if (list[i].type == is_vegie && list[i].vegieptr != nullptr) {
            std::cout << "[Vegie] " << list[i].vegieptr->getName()
                      << " | PHP " << list[i].vegieptr->getPrice()
                      << " | Quantity: x" << list[i].vegieptr->getQuantity()
                      << " | Subtotal: PHP " << list[i].vegieptr->CalcTotalvegiePrice() << std::endl;
        }
    }
}
```

```cpp
int main() {
    const int listsize = 4;

    std::cout << "Grocery List\n" << std::endl;

    // problem 2 using of array of structures to store the grocery items
    groceryitem GroceryList[listsize];
    GroceryList[0] = { is_fruit, new fruits("Apple", 10.0, 7), nullptr };
    GroceryList[1] = { is_fruit, new fruits("Banana", 10.0, 8), nullptr };
    GroceryList[2] = { is_vegie, nullptr, new vegies("Broccoli", 60.0, 12) };
    GroceryList[3] = { is_vegie, nullptr, new vegies("Lettuce", 50.0, 10) };

    // Displaying all details about saved instances
    DisplayGroceryList(GroceryList, listsize);

    std::cout << "\nTotal Sum: PHP " << TotalSum(GroceryList, listsize) << std::endl;
    std::cout << "Initial Total Sum: PHP " << TotalSum(GroceryList, listsize) << std::endl;

    // problem 4 deleting the instance of Lettuce and freeing up memory
    std::cout << "\n Deleting Lettuce" << std::endl;
    for (int i = 0; i < listsize; ++i) {
        if (GroceryList[i].type == is_vegie && GroceryList[i].vegieptr != nullptr) {
            if (GroceryList[i].vegieptr->getName() == "Lettuce") {
                delete GroceryList[i].vegieptr;        // De-allocate memory
                GroceryList[i].vegieptr = nullptr;    // Clear the pointer
                std::cout << "\n-> Lettuce has been deleted and memory freed.\n" << std::endl;
            }
        }
    }

    // Display updated details and updated sum
    DisplayGroceryList(GroceryList, listsize);
    std::cout << "Updated Total Sum: PHP " << TotalSum(GroceryList, listsize) << std::endl;

    // Cleaning up remaining dynamic memory before program termination
    std::cout << "\nCleaning Up Remaining Memory dont turn off system" << std::endl;
    for (int i = 0; i < listsize; ++i) {
        if (GroceryList[i].type == is_fruit && GroceryList[i].fruitptr != nullptr) {
            delete GroceryList[i].fruitptr;
            GroceryList[i].fruitptr = nullptr;
        }
        else if (GroceryList[i].type == is_vegie && GroceryList[i].vegieptr != nullptr) {
            delete GroceryList[i].vegieptr;
            GroceryList[i].vegieptr = nullptr;
        }
    }

    return 0;
}
```

OUTPUT

```
Grocery List

The constructor is called.
The constructor is called.
The constructor is called.
The constructor is called.

 GROCERY LIST
[Fruit] Apple | PHP 10 | Quantity: x7 | Subtotal: PHP 70
[Fruit] Banana | PHP 10 | Quantity: x8 | Subtotal: PHP 80
[Vegie] Broccoli | PHP 60 | Quantity: x12 | Subtotal: PHP 720
[Vegie] Lettuce | PHP 50 | Quantity: x10 | Subtotal: PHP 500

Total Sum: PHP 1370
Initial Total Sum: PHP 1370

 Deleting Lettuce
The Deconstuctor is called.

-> Lettuce has been deleted and memory freed.


 GROCERY LIST
[Fruit] Apple | PHP 10 | Quantity: x7 | Subtotal: PHP 70
[Fruit] Banana | PHP 10 | Quantity: x8 | Subtotal: PHP 80
[Vegie] Broccoli | PHP 60 | Quantity: x12 | Subtotal: PHP 720
Updated Total Sum: PHP 870

Cleaning Up Remaining Memory dont turn off system
The Deconstuctor is called.
The Deconstuctor is called.
The Deconstuctor is called.
```

Observation & Conclusion:

During this activity I observed that by creating 2 separate classes which are fruits and veggies where each item will store each name, prices and quantity. In order to fit both types into one array, a structure groceryitem was used to hold the pointers for either class. When the program creates a new item the constructor will run automatically the same goes with when the lettuce was removed.

In conclusion, I created two separate classes for fruits and veggies so each item could hold its own name, price, and quantity. I then put them together inside a single array using a labeled structure that holds a pointer to either item type. The program then calculates the total price for all items when they are created using dynamic memory. When an item is deleted for example the lettuce was deleted, then the

code frees up a memory and automatically calculates the new total without the lettuce. Then in the end this will automatically delete all the files thanks to the deconstructor and frees up memory.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, ideas generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **At first I asked for gemini to explain the step by step process and how to manually code the questions without giving me the actual code, then I was stuck at how to continue the code after creating the classes so I ask gemini to show me what syntaxes i should do but with out giving the straight code for all of the code,**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.